PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 03 -

MÓDULOS, LOOPS E CONDICIONAIS

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 6
MERCADO, CASES E TENDÊNCIAS 14
O QUE VOCÊ VIU NESTA AULA? 15
REFERÊNCIAS 16

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Você já percebeu que, à medida que a infraestrutura cresce, manter o código organizado e sem repetição se torna um desafio. É aqui que entram três ferramentas poderosas do Terraform: módulos, loops e condicionais. Nesta aula, vamos aprender a criar módulos reutilizáveis, que funcionam como peças de Lego de infraestrutura, prontos para serem encaixados em diferentes projetos. Também exploraremos os repositórios de módulos públicos, como o Terraform Registry e o TofuHub, que aceleram a entrega de soluções já testadas pela comunidade.

Mas não para por aí. Vamos dominar o uso de loops, com `count` e `for_each`, para gerar múltiplos recursos dinamicamente sem duplicar código. E, para fechar, entraremos nas condicionais que dão inteligência ao projeto, permitindo aplicar regras e exceções com expressões como `ternary`, `can` e `try`. Ao final da aula, você terá uma caixa de ferramentas muito mais rica para construir projetos escaláveis, flexíveis e prontos para crescer com segurança.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

Nesta etapa, você vai acompanhar como transformar código Terraform em algo modular, dinâmico e inteligente. O foco é mostrar na prática como quebrar grandes projetos em módulos reutilizáveis, como usar loops para criar múltiplos recursos de forma eficiente e como aplicar condicionais que deixam sua infraestrutura mais adaptável. Você verá como pequenos ajustes no código fazem toda a diferença na hora de reduzir repetição e aumentar a clareza.

Nos vídeos, vamos começar organizando um módulo de rede simples, que depois poderá ser usado em diferentes ambientes. Em seguida, partiremos para os loops, explorando os blocos `count` e `for_each` para provisionar múltiplos recursos de uma só vez. Imagine precisar criar três instâncias EC2 idênticas: com loops, isso se resolve em apenas algumas linhas. Já nas condicionais, você verá como expressões lógicas ajudam a ativar ou desativar recursos de acordo com variáveis, sem precisar duplicar código.

Para ilustrar, aqui vão dois exemplos simples que veremos no vídeo. O primeiro mostra o uso de `count` para criar várias instâncias EC2 de uma só vez:

```
resource "aws_instance" "web" {
  count         = 3
  ami           = var.ami_id
  instance_type = “t2.micro”

  tags = {
    Name = “web-${count.index}"
  }
}
```

No segundo exemplo, uma condicional decide o tipo de instância com base em uma variável booleana:

```
variable "is_production" {
  type    = bool
  default = false
}

resource "aws_instance" "app" {
  ami           = var.ami_id
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
      instance_type   =   var.is_production   ?   "t3.large"   :
“t2.micro"
    }
```

Esses exemplos simples já mostram o poder dos loops e condicionais. Ao longo da aula, você aprenderá a aplicá-los em contextos mais complexos e a integrar esses recursos em módulos reutilizáveis, preparando seu código para crescer em escala sem perder a organização.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

Quando o código de infraestrutura cresce, duas perguntas aparecem rápido: “como evitar repetir meu código?” e “como reutilizar o mesmo código em diferentes contextos sem virar um emaranhado de `if`s?”. A resposta, no ecossistema Terraform/OpenTofu, vem de três pilares que trabalham em conjunto: módulos (organização e reuso), loops (repetição dinâmica) e condicionais (comportamento inteligente). Nesta seção, vamos detalhar cada um desses recursos, sempre intercalando teoria e prática com exemplos reais que você poderá executar. Sempre que quiser ver tudo fluindo na prática, volte às aulas: pause, replique e compare seus resultados revisando os trechos de código.

Módulos: organizando e reaproveitando infraestrutura

Um módulo no Terraform é uma unidade de reuso: uma pasta contendo arquivos `.tf` que descrevem recursos específicos. Módulos são úteis porque encapsulam lógica, reduzem repetição e trazem clareza para times que trabalham em projetos grandes. Pense neles como “peças de Lego” de rede, computação, banco de dados e observabilidade, que você encaixa de forma previsível. Um módulo saudável isola uma responsabilidade (por exemplo, “rede VPC + subnets”), recebe variáveis tipadas e documentadas, expõe outputs úteis (IDs, CIDRs, endpoints) e evita acoplar decisões de ambiente; isso fica para o código “raiz” (root module).

Veja este exemplo de módulo de rede que cria uma VPC e múltiplas subnets dinamicamente:

modules/network/main.tf

```
# Criação da VPC principal
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
  tags = {
    Name = "main-vpc"
  }
}
# Criação dinâmica de subnets usando for_each
resource "aws_subnet" "subnets" {
  for_each          = var.subnets
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value
  availability_zone = each.key
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
    tags = {
      Name = "subnet-${each.key}"
    }
  }
```

modules/network/variables.tf

```
variable "vpc_cidr" {
  description = "CIDR da VPC"
  type        = string
}

variable "subnets" {
  description = "Mapa de zonas de disponibilidade e seus CIDRs"
  type        = map(string)
}
```

modules/network/outputs.tf

```
output "vpc_id" {
  description = "ID da VPC criada"
  value       = aws_vpc.main.id
}

output "subnet_ids" {
  description = "Lista de IDs das subnets"
  value       = values(aws_subnet.subnets)[*].id
}
```

Esse módulo recebe variáveis em `variables.tf` e retorna outputs em `outputs.tf`. No projeto raiz, consumimos o módulo assim:

Uso no módulo raiz:

```
module "network" {
  source   = "./modules/network"
  vpc_cidr = "10.0.0.0/16"
  subnets = {
    "us-east-1a" = "10.0.1.0/24"
    "us-east-1b" = "10.0.2.0/24"
  }
}
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

A organização mínima em projetos grandes passa por um diretório `modules/` com componentes reutilizáveis e um `live/` para ambientes (dev, staging, prod) que orquestram esses módulos. Ao consumir, o `source` do módulo pode apontar para um caminho local (`./modules/vpc`), um repositório Git (com tag para travar versão) ou um registro público (Terraform Registry / TofuHub) com versionamento semântico (`~> 1.2`). A cada consumo, documente claramente o que entra (variáveis), o que sai (outputs) e o que o módulo NÃO faz; isso reduz mal-entendidos.

Repositórios de Módulos: Terraform Registry e TofuHub

Um dos grandes benefícios de trabalhar com módulos é que você não precisa começar do zero. A HashiCorp mantém o Terraform Registry, e a comunidade OpenTofu oferece o TofuHub — repositórios públicos com milhares de módulos prontos para uso. Esses módulos são publicados e versionados por fornecedores oficiais, empresas e pela comunidade, cobrindo desde VPCs, bancos de dados e clusters Kubernetes até serviços de monitoramento e segurança.

A ideia é simples: você pode importar um módulo público em vez de escrever tudo manualmente. Isso economiza tempo, padroniza boas práticas e ainda permite aprender com a forma como a comunidade resolve problemas reais. Claro, é importante auditar antes de usar em produção, mas para protótipos ou como inspiração, esses repositórios são excelentes.

Veja um exemplo prático consumindo um módulo do Terraform Registry para criar uma VPC:

```
    module "vpc" {
      source  = "terraform-aws-modules/vpc/aws"
      version = "5.0.0"

      name = "my-vpc"
      cidr = "10.0.0.0/16"

      azs             = ["us-east-1a", "us-east-1b", "us-east-
1c"]
      private_subnets    =    ["10.0.1.0/24",    "10.0.2.0/24",
"10.0.3.0/24"]
      public_subnets    =  ["10.0.101.0/24",  "10.0.102.0/24",
"10.0.103.0/24"]

      enable_nat_gateway = true
    }
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Nesse caso, todo o trabalho de criar subnets, rotas e NAT Gateways já está encapsulado no módulo. Basta informar variáveis como `cidr`, `azs` e subnets.

O TofuHub funciona de forma similar, sendo a alternativa da comunidade OpenTofu. O fluxo é o mesmo: você importa o módulo com `source` apontando para o repositório (Git, Registry ou TofuHub), define as variáveis necessárias e reutiliza o código.

Em resumo, enquanto módulos locais ajudam a organizar seu projeto e a compartilhar dentro do time, módulos públicos ampliam ainda mais a eficiência, trazendo soluções validadas pela comunidade e acelerando a entrega de infraestrutura.

Loops: automatizando criação em massa

Entrando nos loops, o objetivo é gerar muitos recursos sem repetir blocos. O Terraform oferece dois caminhos: `count` e `for_each`. `Count` é ótimo para "N iguais" (por exemplo, três instâncias idênticas), mas indexa por posição; se você apagar o item do meio, os índices mudam e o Terraform pode identificar objetos de forma inesperada. Já `for_each` indexa por chaves estáveis (strings do seu conjunto/mapa) e preserva o vínculo recurso↔chave ao longo do tempo, o que tende a produzir planos mais previsíveis. É por isso que, em produção, preferimos `for_each` para qualquer coisa que tenha identidade (subnets nomeadas, regras de segurança, buckets por equipe). Além disso, quando o provedor usa blocos aninhados (como regras de Security Group), `dynamic` blocks com `for_each` deixam o código enxuto: você passa uma coleção de regras e o Terraform renderiza cada bloco; isso evita "copiar/colar". A troca de `count` para `for_each` muda a addressing no state e estabiliza os planos (`aws_instance.web["api"]` em vez de `aws_instance.web[1]`);

Em resumo:

- `count` cria um número fixo de recursos idênticos.
- `for_each` cria recursos indexados por chave, o que é mais seguro quando cada item tem identidade própria.

Veja o mesmo caso (criar instâncias EC2) implementado das duas formas:

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
# Usando count (instâncias iguais)
resource "aws_instance" "web" {
  count         = 2
  ami           = var.ami_id
  instance_type = "t2.micro"

  tags = {
    Name = "web-${count.index}"
  }
}
```

```
# Usando for_each (instâncias diferentes)
variable "instances" {
  type = map(string)
  default = {
    "api"   = "t2.micro"
    "cache" = "t3.small"
  }
}

resource "aws_instance" "apps" {
  for_each      = var.instances
  ami           = var.ami_id
  instance_type = each.value

  tags = {
    Name = "app-${each.key}"
  }
}
```

Perceba como `for_each` preserva a identidade de cada recurso (`app["api"]`, `app["cache"]`), o que evita problemas no state quando um item é removido.

Condicionais: tornando o código inteligente

As condicionais trazem inteligência ao nosso código. A expressão ternária `cond ? A : B` atende a maioria dos casos, mas há padrões que tornam o código ainda mais seguro: usar `null` para "omitir" argumentos quando não aplicável (muitos provedores tratam `null` como "não aplicar"), preferir `for_each` condicional a `count` quando você precisa criar zero ou um recurso (por exemplo, `for_each = var.enable_nat ? toset(["this"]) : toset([])`), e explorar funções

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

como `can()` e `try()` quando há risco de uma chave opcional não existir em um objeto.

Exemplo: mudar o tipo de instância dependendo do ambiente:

```
variable "is_production" {
  type    = bool
  default = false
}

resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type   =   var.is_production   ?   "t3.large"   :
"t2.micro"

  tags = {
    Name = var.is_production ? "prod-app" : "dev-app"
  }
}
```

Podemos ir além, criando recursos inteiros de forma condicional:

```
variable "enable_nat" {
  type    = bool
  default = false
}

resource "aws_nat_gateway" "this" {
  for_each        =  var.enable_nat  ?  toset(["active"])  :
toset([])
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.subnets["us-east-1a"].id
}

resource "aws_eip" "nat" {
  vpc = true
}
```

Aqui, se `enable_nat = false`, nenhum NAT Gateway é criado.

Em módulos, condicionais em variáveis podem servir para realizar validações e ajudam a guiar o usuário, por exemplo:

```
"se is_production = true, então min_size >= 2";
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

A regra de negócio vira código e falhas são identificadas cedo no `plan`.

Um cuidado final: condicionais não devem virar um labirinto — quando a lógica de ativar/desativar começa a se multiplicar, é sinal de que vale separar em módulos menores e compor na raiz.

Para ir além, explore dois recursos que ampliam bastante seu alcance. `for` expressions permitem transformar coleções de forma declarativa — por exemplo, gerar um mapa de tags a partir de uma lista de equipes — sem escrever blocos extras; é comum vê-las compondo `locals` que "preparam" dados para `for_each`. E `dynamic` blocks dentro de recursos aninhados (como `ingress`/`egress` em Security Groups, `listener` em Load Balancers) evitam duplicação e facilitam políticas; você passa um objeto de alto nível (validado em `variables.tf`) e o recurso "decompõe" isso em N blocos. Como estudo extra, tente refatorar um recurso com quatro blocos ingress repetidos para um `dynamic "ingress"` com `for_each` sobre uma lista de portas

Dynamic Blocks: quando recursos têm blocos repetitivos

Muitos recursos da AWS, como Security Groups, exigem blocos aninhados repetidos (ex.: várias regras de ingress). Copiar e colar é um problema. Para isso, usamos `dynamic` blocks.

```
resource "aws_security_group" "web_sg" {
  name   = "web-sg"
  vpc_id = module.network.vpc_id


  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.from_port
      to_port     = ingress.value.to_port
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }


  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
```

Essa técnica recebe uma lista de regras em `variables.tf` e o Terraform monta automaticamente os blocos `ingress`.

For Expressions: coleções mais inteligentes

Outro recurso avançado são as for expressions, que permitem transformar listas e mapas. Um exemplo: gerar tags automaticamente a partir de variáveis.

```
locals {
  instance_tags = { for name, type in var.instances : name
=> "env-${var.environment}-${type}" }
}

output "instance_tags_map" {
  value = local.instance_tags
}
```

Se var.environment = "dev" e var.instances = { api = "t2.micro", cache = "t3.small" }, a saída será:

```
instance_tags_map = {
  "api"   = "env-dev-t2.micro"
  "cache" = "env-dev-t3.small"
}
```

Isso simplifica muito a geração de metadados consistentes entre ambientes.

Conclusão

Módulos, loops, condicionais, dynamic blocks e for expressions formam um conjunto poderoso para estruturar projetos Terraform robustos. A prática recomendada é sempre buscar clareza e previsibilidade: prefira `for_each` a `count` quando recursos têm identidade, documente variáveis, use condicionais com parcimônia e explore dynamic blocks e for expressions para reduzir repetição.

Agora é a sua vez: revise os exemplos, replique-os em sua máquina e acompanhe as videoaulas para ver cada um desses recursos aplicados em cenários reais. Quanto mais você praticar, mais claro ficará como escrever infraestrutura como código de forma enxuta, organizada e inteligente.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

O uso de módulos, loops e condicionais não é só uma prática recomendada: é uma necessidade para empresas que operam com milhares de recursos e times distribuídos. Cada vez mais, grandes organizações vêm estruturando catálogos internos de módulos, onde blocos de rede, compute ou observabilidade podem ser consumidos sob demanda. Essa estratégia traz consistência, acelera o provisionamento e reduz falhas.

No artigo “Infrastructure as Code for the Modern Era”, quatro cases mostram como isso se traduz na prática. A Uber utiliza módulos reutilizáveis para padronizar ambientes em escala global, garantindo consistência mesmo com times diferentes trabalhando em paralelo. O Slack, por sua vez, encontrou nos loops e condicionais uma forma de gerenciar infraestrutura multirregião de maneira previsível, sem duplicar código. Já a Coinbase aproveita a modularização para reforçar padrões de segurança em sua infraestrutura crítica, reduzindo riscos em operações financeiras. E o Shopify, com sua enorme base de serviços, utiliza condicionais para adaptar recursos de acordo com sazonalidade e picos de demanda, mantendo eficiência operacional.

Olhando para frente, a tendência é clara: a infraestrutura será cada vez mais tratada como produto interno, com times de plataforma fornecendo módulos versionados e documentados, enquanto loops e condicionais asseguram flexibilidade para diferentes cenários. Complementando, políticas automatizadas de segurança (policy as code) garantem que essa flexibilidade aconteça sem comprometer conformidade e boas práticas.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu como estruturar módulos reutilizáveis, capazes de organizar e padronizar a infraestrutura em projetos de diferentes tamanhos. Viu também como os loops (`count` e `for_each`) ajudam a criar múltiplos recursos de forma dinâmica, reduzindo a repetição de código e garantindo maior previsibilidade no state.

Exploramos o uso de condicionais, que permitem tornar sua infraestrutura mais inteligente e adaptável, ativando ou desativando recursos conforme o ambiente. Além disso, você conheceu dois recursos avançados: os dynamic blocks, que automatizam a criação de blocos repetitivos (como regras de segurança), e as for expressions, que transformam coleções de forma elegante e consistente.

Agora você tem à disposição um conjunto de práticas e técnicas que permitem escrever código Terraform mais enxuto, flexível e sustentável — pronto para escalar junto com as necessidades do seu time e da sua empresa.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

HASHICORP. **Terraform Registry.** HashiCorp, 2025. Disponível em: https://registry.terraform.io/. Acesso em: 21 out. 2025.

OPENTOFU. **Docs. OpenTofu**. 2025. Disponível em: https://opentofu.org/docs/. Acesso em: 21 out. 2025.

HASHICORP. **Meta-arguments.** HashiCorp Learn. 2025. Disponível em: https://developer.hashicorp.com/terraform/language/meta-arguments/. Acesso em: 21 out. 2025.

HASHICORP. **Conditionals.** HashiCorp Learn. 2025. Disponível em: https://developer.hashicorp.com/terraform/language/expressions/conditionals. Acesso em: 21 out. 2025.

HASHICORP. **Dynamic Blocks.** HashiCorp Learn. 2025. Disponível em: https://developer.hashicorp.com/terraform/language/expressions/dynamic-blocks. Acesso em: 21 out. 2025.

HASHICORP. **For Expressions.** HashiCorp Learn. 2025. Disponível em: https://developer.hashicorp.com/terraform/language/expressions/for. Acesso em: 21 out. 2025.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

terraform. loops. Condicionais, modularidade.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com